

탐구 - 녹는점 예측



탐구

분자의 녹는점은 분자간 힘(수소결합, 극성, 분산력)과 구조적 요인(대칭성)에 의해 결정된다는 것이 화학적인 정설이다. 이번 탐구는 이러한 화학적 요인이 실제 녹는점 예측 모델에서 얼마나 유의미하게 작용하는지 정량적으로 검증하고자 하였다.

탐구 동기

물질의 녹는점은 화학 교과에서 분자간 힘의 세기를 설명하는 대표적 지표로 다뤄지지만, 정작 '어떤 요인이 얼마나 결정적으로 작용하는가'는 정성적으로만 서술되는 경우가 많다. 이를 정량적으로 검증하고 싶었고, 마침 진행 중이던 머신러닝 프로젝트 경험을 활용해 데이터 기반으로 이 질문에 접근하고자 이번 탐구를 기획하였다.

1. 데이터 수집

데이터셋 : [melting_point](#)

- 데이터셋은 이번 년도 초에 대회가 이루어졌던 [Thermophysical Property: Melting Point](#)의 데이터셋을 사용하였다.

```
1 import kagglehub
2
3 # Download latest version
4 path = kagglehub.competition_download('melting-point')
5
6 print("Path to competition files:", path)
```

kagglehub를 이용해서 다운

작업 폴더로 mv 하였다.

```
1 mv 원본경로 목적경로
```

2. 데이터 구조

- 데이터셋 구조

```
1 from pathlib import Path
2
3 path = Path("폴더경로")
4
5 for p in sorted(path.iterdir()):
6     kind = "폴더" if p.is_dir() else "파일"
7     print(kind, "|", p.name)
```

데이터셋의 구조는 다음 코드를 통해 확인 할 수 있다.

파 일		sample_submission.csv
파 일		test.csv
파 일		train.csv

폴더는 모두 파일로 구성되어 있고

각 csv 파일의 역할은

train - 전체 칼럼 존재

test - tm(녹는점)이 없음

sample_submission - id,tm만 있음

이를 통해 train을 통해 모델을 학습시키고 test로 예측한 tm을 id와 함께 saple_submission에 넣으면 됨

```
1 import pandas as pd
2
3 df = pd.read_csv("csv파일 경로")
4 print(df.shape) # (행 수, 열 수)
5 print(df.describe()) # 수치형 컬럼 통계 요약 (min/max/mean etc.)
```

출력 결과

(2662, 427)										
	id	Tm	Group 1	Group 2	...	Group 421	Group 422	Group 423	Group 424	
count	2662.000000	2662.000000	2662.000000	2662.000000	...	2662.0	2662.0	2662.0	2662.0	
mean	1652.887303	278.263452	1.020661	1.582269	...	0.0	0.0	0.0	0.0	
std	955.005512	85.117914	1.353640	3.738416	...	0.0	0.0	0.0	0.0	
min	1.000000	53.540000	0.000000	0.000000	...	0.0	0.0	0.0	0.0	
25%	830.250000	217.000000	0.000000	0.000000	...	0.0	0.0	0.0	0.0	
50%	1642.500000	277.300000	1.000000	0.000000	...	0.0	0.0	0.0	0.0	
75%	2469.750000	325.150000	2.000000	1.000000	...	0.0	0.0	0.0	0.0	
max	3328.000000	897.150000	16.000000	42.000000	...	0.0	0.0	0.0	0.0	

행 2662 열 427로
학습 데이터에 포함되는 분자 수는 2662개이다.

분자식은 SMILES로 작성되어있다.

SMILES란?

복잡한 화학 분자 구조를 컴퓨터가 이해할 수 있는 문자열 형태로 표현하는 표기법

- 1. 대문자 = 지방족, 소문자 = 방향족
- 2. 수소는 보통 생략된다
- 3. 숫자 = 고리 열림/닫힘 표시
- 4. 대괄호 = 특수한 원자 상태

3. 데이터 분석

1.

```
1 import pandas as pd
2
3 df = pd.read_csv("데이터 경로")
4
5 print(df["Tm"].describe()) # Tm 범위 보고 단위가 K인지 °C인지 판단해보기
6 group_cols = [c for c in df.columns if c.startswith("Group")]
7 print(df[group_cols].values.max()) # Group 값 최댓값 → 카운트인지 binary인지
```

출력 결과

count	2662.000000
mean	278.263452
std	85.117914
min	53.540000
25%	217.000000
50%	277.300000
75%	325.150000
max	897.150000
Name: Tm, dtype: float64	
42	

tm의 최솟값이 53.54임을 통해 tm이 °C아닌 K임을 알 수 있고
Group이 0/1 로 존재,존재 안함이 아니라 어떤 개수를 나타냄을 알 수 있다.

2.

```
1 import pandas as pd
2
3 df = pd.read_csv("/Users/choejihu/Documents/G-code/melting/dataset/melting-point/train.csv")
4
5 group_cols = [c for c in df.columns if c.startswith("Group")]
6 zero = [c for c in group_cols if df[c].sum() == 0]
7 print(f"항상 0인 컬럼: {len(zero)}개")
```

출력 결과

항 상 0인 컬 럼 : 87개

즉, 데이터셋에서 87개 칼럼은 아무것도 없다는 것

3.

```
1 import pandas as pd
2
3 df = pd.read_csv("데이터 경로")
4
5 group_cols = [c for c in df.columns if c.startswith("Group")]
6
7
8 active = [c for c in group_cols if df[c].sum() > 0]
9 corr = df[active].corrwith(df["Tm"]).sort_values(key=abs, ascending=False)
10 print(corr.head(10)) # 어떤 Group이 Tm과 연관 높은지 직접 보기
```

0이 아닌 칼럼의 피어슨 상관계수를 보았을 때

Group 15	0.387580
Group 18	0.223551
Group 16	0.223530
Group 401	0.222525
Group 17	0.194420
Group 1	-0.186595
Group 123	0.179178
Group 365	0.176346
Group 31	0.171773
Group 412	0.165621
dtype: float64	

생각보다 단일 칼럼의 상관계수가 낮다는 것을 알 수 있다.

4.

```
1 import pandas as pd
2
3 from sklearn.linear_model import LinearRegression
4
5
6
7 df = pd.read_csv("/Users/choejihu/Documents/G-code/melting/dataset/melting-point/train.csv")
8
9 group_cols = [c for c in df.columns if c.startswith("Group")]
10 active = [c for c in group_cols if df[c].sum() > 0]
11 corr = df[active].corrwith(df["Tm"]).sort_values(key=abs, ascending=False)
12
13 for n in [5, 10, 20, 50]:
14     topn = corr.head(n).index.tolist()
15
16     X = df[topn].values
17     r2 = LinearRegression().fit(X, df["Tm"]).score(X, df["Tm"])
18     print(f"상위 {n}개 → R² = {r2:.3f}")
```

각 상위 그룹끼리 묶었을 때 선형회귀 해보면

상 위 5개	→ R² = 0.179
상 위 10개	→ R² = 0.255
상 위 20개	→ R² = 0.359
상 위 50개	→ R² = 0.507

과적합을 추정 가능하다.
(그룹을 더 묶을 수록 상관계수 올라감)

학습

```
1 import pandas as pd
2 import lightgbm as lgb
3 import joblib
4 from sklearn.model_selection import cross_val_score
5
6
7 TRAIN_PATH = "학습 데이터 경로"
8 MODEL_PATH = "모델 저장 경로"
9
10
11 def load_data(path):
12     df = pd.read_csv(path)
```

```
13     group_cols = [c for c in df.columns if c.startswith("Group")]
14     active_cols = [c for c in group_cols if df[c].sum() > 0]
15     X = df[active_cols]
16     y = df["Tm"]
17     return X, y, active_cols
18
19
20 def build_model():
21     return lgb.LGBMRegressor(
22         n_estimators=500,
23         learning_rate=0.05,
24         num_leaves=31,
25         random_state=42,
26         verbose=-1,
27     )
28
29
30 def evaluate(model, X, y):
31     r2 = cross_val_score(model, X, y, cv=5, scoring="r2").mean()
32     mae = -cross_val_score(model, X, y, cv=5, scoring="neg_mean_absolute_error").mean()
33     return r2, mae
34
35
36 def show_importance(model, feature_names, top_n=10):
37     ranking = sorted(zip(feature_names, model.feature_importances_),
38                      key=lambda pair: pair[1], reverse=True)
39     print(f"\n예측에 가장 영향을 준 피쳐 TOP {top_n}:")
40     for name, score in ranking[:top_n]:
41         print(f"  {name:12s} {score}")
42
43
44 def main():
45     X, y, feature_names = load_data(TRAIN_PATH)
46     print(f"분자 {len(X)}개, 피쳐 {len(feature_names)}개")
47
48     model = build_model()
49
50     r2, mae = evaluate(model, X, y)
51     print(f"교차검증 성능 -> R² = {r2:.3f}, MAE = {mae:.2f} K")
52
53     model.fit(X, y)
54     print("최종 모델 학습 완료")
55
56     joblib.dump({"model": model, "features": feature_names}, MODEL_PATH)
57     print(f"모델 저장 -> {MODEL_PATH}")
58
59     show_importance(model, feature_names)
60
61
62 if __name__ == "__main__":
63     main()
```

R² = 0.533, MAE = 40.28 K

결과

예측에 가장 영향을 준 피쳐 TOP 10:

Group 2	2378
Group 15	1407
Group 1	1382
Group 3	521
Group 168	496
Group 123	471
Group 20	430
Group 31	381
Group 41	307
Group 29	289

3번에서 상관계수가 낮았다는 것을 바탕으로 각 칼럼끼리 연관지어 학습을 시켜야겠다는 가설을 세웠고 이를 실험하기 위해 lightgbm 을 골랐다.(칼럼 중요도 분석을 위해)
이를 통해 칼럼들 중 결과에 영향을 많이 주는 것들을 뽑아낼 수 있었다.
그러나 상관계수가 아직 0.533 정도이기에 이를 기반으로 개선하는 모델이 필요하다.

각 그룹이 무엇을 의미하는지 알기 위해 RDKit 라이브러리와 대조해보기로 한다.

그룹 분석

```
1  import pandas as pd
2  import numpy as np
3  import joblib
4  from rdkit import Chem
5  from rdkit.Chem import Descriptors, Fragments
6  from rdkit import RDLogger
7
8  RDLogger.DisableLog("rdApp.*")
9
10 TRAIN_PATH = "학습 데이터 경로"
11 MODEL_PATH = "모델 저장 경로"
12 TOP_N = 10
13
14
15 def load_model_and_top_features(model_path, top_n):
16     saved = joblib.load(model_path)
17     model = saved["model"]
18     features = saved["features"]
19     ranking = sorted(zip(features, model.feature_importances_),
20                      key=lambda pair: pair[1], reverse=True)
21     return ranking[:top_n]
22
23
24 def build_candidate_table(smiles_list):
25     mols = [Chem.MolFromSmiles(s) for s in smiles_list]
26     candidates = {}
27     for name, func in Fragments.__dict__.items():
28         if name.startswith("fr_"):
29             try:
30                 candidates[name] = [func(m) for m in mols]
31             except Exception:
32                 pass
33     candidates["aromatic_atoms"] = [sum(1 for a in m.GetAtoms() if a.GetIsAromatic()) for m in mols]
34     candidates["ring_count"] = [Descriptors.RingCount(m) for m in mols]
35     candidates["carbon_count"] = [sum(1 for a in m.GetAtoms() if a.GetSymbol() == "C") for m in mols]
36     return pd.DataFrame(candidates)
37
38
39 def match_group(group_values, candidate_table):
40     best_name, best_r = None, 0.0
41     for cand_name in candidate_table.columns:
42         cand_values = candidate_table[cand_name].values
43         if cand_values.std() == 0:
44             continue
45         r = np.corrcoef(group_values, cand_values)[0, 1]
46         if abs(r) > abs(best_r):
47             best_r, best_name = r, cand_name
48     exact = (group_values == candidate_table[best_name].values).mean()
49     return best_name, best_r, exact
50
51
52 def main():
53     df = pd.read_csv(TRAIN_PATH)
54     top_features = load_model_and_top_features(MODEL_PATH, TOP_N)
55     candidate_table = build_candidate_table(df["SMILES"])
56
57     print(f"중요도 TOP {TOP_N} Group의 정체 (RDKit 작용기 대조):\n")
58     print(f"{'Group':11s} {'중요도':>6s} {'추정 작용기':18s} {'상관':>7s} {'일치율':>6s} 판정")
59     print("-" * 70)
60
61     for group_name, importance in top_features:
62         group_values = df[group_name].values
63         cand_name, r, exact = match_group(group_values, candidate_table)
64         if exact > 0.9:
65             verdict = "확실"
66         elif abs(r) > 0.7:
67             verdict = "유력"
68         else:
69             verdict = "불명(고차그룹 추정)"
70         print(f"{'group_name':11s} {'importance':6d} {'cand_name':18s} {'r':+.3f} {'exact*100:5.0f}%
71 {'verdict}"))
```

```
71
72
73     if __name__ == "__main__":
74         main()
```

출력 결과

Group	중요도	추정	작용기	상관	일치율	판정
Group 2	2378	fr_unbrch_alkane	+0.961	66%	유력	
Group 15	1407	aromatic_atoms	+0.887	51%	유력	
Group 1	1382	ring_count	-0.310	23%	불명	(고차그룹 추정)
Group 3	521	fr_Al_OH_noTert	+0.253	88%	불명	(고차그룹 추정)
Group 168	496	fr_piperdine	+0.341	90%	확실	
Group 123	471	fr_halogen	+0.310	79%	불명	(고차그룹 추정)
Group 20	430	fr_aryl_methyl	+0.749	93%	확실	
Group 31	381	fr_Al_COO	+0.995	100%	확실	
Group 41	307	fr_ester	+0.581	94%	확실	
Group 29	289	fr_Al_OH	+0.939	99%	확실	

RDKit이란?

RDKit은 분자를 다루는 파이썬 라이브러리로서 화학자,연구자들이 쓰는 오픈소스 도구이다.
SMILES 문자열을 진짜 "분자 객체"로 바꿔서, 그 분자의 성질을 계산해준다.

예를 들어 "CCO" (에탄올)라는 글자를 주면, RDKit이 이걸 "탄소-탄소-산소로 연결된 분자"로 이해하고

- 분자량이 얼마인지 (Descriptors.MolWt)
- -OH가 몇 개인지 (Fragments.fr_Al_OH)
- 방향족 고리가 있는지
- 벤젠 고리가 몇 개인지

이런 걸 자동으로 세준다.

이를 활용하여 RDKit의 결과와 현재 train.csv 파일의 그룹별 개수를 대조하여 역추론이 가능하다.

그래서 결과를 분석해보자면

Group 2	분지 없는 알케인 사슬 (-CH ₂ -)	분산력 (사슬 길이)
Group 15	방향족 원자	결정 안정성 (평균 구조)
Group 168	피페리딘 (고리형 아민)	고리 구조
Group 20	방향족 메틸 (아릴-CH ₃)	방향족 치환
Group 31	지방족 카복실산 (-COOH)	강한 수소결합 (이합체)
Group 41	에스터 (-COO-)	극성
Group 29	지방족 알코올 (-OH)	수소결합

이러한 결과를 얻을 수 있다.

순위	Group (정체)	모델 중요도	실제 화학에서 녹는점 영향	실제로 맞는지?
1	Group 2 (알케인 사슬)	2378	분산력 — 있지만 지배적이진 않음	△
2	Group 15 (방향족)	1407	매우 큼 (평균→결정 안정)	✓
3	Group 1 (불명)	1382	?	?
4	Group 3 (알코올?)	521	수소결합	✓
5	Group 168 (피페리딘)	496	고리 구조	✓
6	Group 123 (할로겐)	471	애매	△
7	Group 20 (방향족 메틸)	430	방향족 관련	✓
8	Group 31 (카복실산)	381	강함 (이합체 수소결합)	✓
9	Group 41 (에스터)	307	극성	✓
10	Group 29 (알코올)	289	수소결합	✓

대부분이 맞는 예측이었지만 이 실험에서 1번 중요도를 가진 그룹2의 경우 실제 화학 이론에서는 크게 상관이 없었던 알케인 사슬이었다.

왜 Group 2가 중요도 1위인가

Group 2(탄소 사슬)가 왜 중요도 1위로 나왔는지 세 가지 가설을 세워 확인하였다.

가설 A: 값의 범위가 넓어서

사슬 길이는 1개부터 40개까지 값이 다양하다. 모델이 데이터를 나눌 때 값의 종류가 많은 피쳐일수록 나눌 지점이 많아 자주 쓴다. 방향족(0~몇 개)보다 값이 다양해서 더 자주 쓰였을 수 있다.

가설 B: 거의 모든 분자에 있어서

탄소 사슬은 유기분자 대부분에 들어있다. 반면 카복실산은 일부 분자에만 있다. 흔한 피쳐일수록 여러 분자를 구분하는 데 자주 쓰인다.

가설 C: 다른 피쳐와 같이 작용해서

사슬 길이가 기본 바탕이 되고, 거기에 어떤 작용기가 붙느냐로 녹는점이 정해진다. 그래서 모델이 사슬로 먼저 나눈 뒤 작용기로 더 나누는 방식일 수 있다.

확인 결과

가설 A는 부분적으로 맞았다. Group 2는 값의 종류가 35개, 최댓값 42로 다른 피쳐보다 범위가 넓다. 자주 쓰일 조건은 맞다. 하지만 이게 전부는 아니었다.

가설 B는 틀렸다. Group 2는 전체 분자의 35%에만 있는데, 오히려 Group 1(53%)과 Group 15(46%)가 더 흔했다. 그러니 "흔해서 1위"는 아니다. 이 가설은 버렸다.

가설 C가 확실히 맞았고, 이게 핵심이다. Group 2를 빼고 학습하면 성능(R^2)이 0.533에서 0.457로 떨어진다. 반면 방향족(Group 15)을 빼면 0.036만 떨어진다. 즉 Group 2를 빼면 방향족을 뺄 때보다 2배 이상 성능이 나빠진다. 단순히 자주 쓰인 게 아니라 실제로 중요한 정보를 담고 있다는 뜻이다.

여기에 더해 재미있는 사실도 찾았다. 사슬이 길어질 때 녹는점이 어떻게 변하는지를, 방향족이 있는 분자와 없는 분자로 나눠서 봤다.

- 방향족이 있는 분자: 사슬이 길수록 녹는점이 22.6K 낮아짐
- 방향족이 없는 분자: 사슬이 길수록 녹는점이 1.3K 낮아짐 (거의 변화 없음)

같은 사슬 길이인데 방향족이 있느냐 없느냐에 따라 효과가 약 17배나 차이 났다. 이는 사슬 길이가 혼자 작용하는 게 아니라 방향족과 같이 작용한다는 것을 보여준다. 긴 사슬이 방향족 분자가 촘촘하게 쌓이는 것을 방해해서 녹는점을 크게 낮추는 것으로 보인다.

개선 방안

우선 RDKit을 이용하여 SMILES 분자식들에 대한 정보들을 추가로 가져온다.

-> SMILES 문자열을 넣으면 분자량,극성과 같은 숫자 데이터를 받아볼 수 있음

```
1  import pandas as pd
2  import lightgbm as lgb
3  import joblib
4  from rdkit import Chem
5  from rdkit.Chem import Descriptors
6  from rdkit import RDLogger
7  from sklearn.model_selection import cross_val_score
8
9  RDLogger.DisableLog("rdApp.*")
10
11 TRAIN_PATH = "학습 데이터 경로"
12 MODEL_PATH = "모델 저장 경로"
13
14
15 def get_descriptors(smiles):
16     mol = Chem.MolFromSmiles(smiles)
17     if mol is None:
18         return None
19     return {
20         "MolWt": Descriptors.MolWt(mol),
21         "HeavyAtom": Descriptors.HeavyAtomCount(mol),
22         "NumHDonors": Descriptors.NumHDonors(mol),
23         "NumHAcceptors": Descriptors.NumHAcceptors(mol),
24         "TPSA": Descriptors.TPSA(mol),
25         "MolLogP": Descriptors.MolLogP(mol),
26         "RotBonds": Descriptors.NumRotatableBonds(mol),
27         "AromRings": Descriptors.NumAromaticRings(mol),
28         "FracCSP3": Descriptors.FractionCSP3(mol),
29         "RingCount": Descriptors.RingCount(mol),
30     }
31
```

```
32
33 def load_data(path):
34     df = pd.read_csv(path)
35     feats = df["SMILES"].apply(get_descriptors)
36     valid = feats.notna()
37     X = pd.DataFrame([f for f in feats if f is not None])
38     y = df.loc[valid, "Tm"].reset_index(drop=True)
39     return X, y, list(X.columns)
40
41
42 def build_model():
43     return lgb.LGBMRegressor(
44         n_estimators=500,
45         learning_rate=0.05,
46         num_leaves=31,
47         random_state=42,
48         verbose=-1,
49     )
50
51
52 def evaluate(model, X, y):
53     r2 = cross_val_score(model, X, y, cv=5, scoring="r2").mean()
54     mae = -cross_val_score(model, X, y, cv=5, scoring="neg_mean_absolute_error").mean()
55     return r2, mae
56
57
58 def show_importance(model, feature_names, top_n=10):
59     ranking = sorted(zip(feature_names, model.feature_importances_),
60                      key=lambda pair: pair[1], reverse=True)
61     print(f"\n예측에 가장 영향을 준 descriptor TOP {top_n}:")
62     for name, score in ranking[:top_n]:
63         print(f"    {name:14s} {score}")
64
65
66 def main():
67     X, y, feature_names = load_data(TRAIN_PATH)
68     print(f"분자 {len(X)}개, descriptor {len(feature_names)}개")
69     print(f"피처: {feature_names}")
70
71     model = build_model()
72
73     r2, mae = evaluate(model, X, y)
74     print(f"\n교차검증 성능 -> R² = {r2:.3f}, MAE = {mae:.2f} K")
75
76     model.fit(X, y)
77     print("최종 모델 학습 완료")
78
79     joblib.dump({"model": model, "features": feature_names}, MODEL_PATH)
80     print(f"모델 저장 -> {MODEL_PATH}")
81
82     show_importance(model, feature_names)
83
84
85 if __name__ == "__main__":
86     main()
```

출력 결과

예 측 에 가 장 영 향 을 준 descriptor TOP 10:	
MolWt	3864
MolLogP	3767
TPSA	1953
HeavyAtom	1413
RotBonds	1336
FracCSP3	1226
RingCount	541
NumHAcceptors	438
NumHDonors	263
AromRings	199

하지만 이것만으로도 정보는 부족하다.
그래서 최종적으로 위에서 뽑은 그룹 칼럼과 조합하여 최종적인 예측을 진행하였다.

결합 피처 347개 = Group 337개 + descriptor 10개

교차검증 성능 -> $R^2 = 0.664$, MAE = 33.36 K

```
1  import pandas as pd
2  import lightgbm as lgb
3  import joblib
4  from rdkit import Chem
5  from rdkit.Chem import Descriptors
6  from rdkit import RDLogger
7  from sklearn.model_selection import cross_val_score
8
9  RDLogger.DisableLog("rdApp.*")
10
11 TRAIN_PATH = "학습 데이터 경로"
12 MODEL_PATH = "모델 저장 경로"
13
14 def get_descriptors(smiles):
15     mol = Chem.MolFromSmiles(smiles)
16     if mol is None:
17         return None
18     return {
19         "MolWt": Descriptors.MolWt(mol),
20         "HeavyAtom": Descriptors.HeavyAtomCount(mol),
21         "NumHDonors": Descriptors.NumHDonors(mol),
22         "NumHAcceptors": Descriptors.NumHAcceptors(mol),
23         "TPSA": Descriptors.TPSA(mol),
24         "MolLogP": Descriptors.MolLogP(mol),
25         "RotBonds": Descriptors.NumRotatableBonds(mol),
26         "AromRings": Descriptors.NumAromaticRings(mol),
27         "FracCSP3": Descriptors.FractionCSP3(mol),
28         "RingCount": Descriptors.RingCount(mol),
29     }
30
31
32 def make_features(df, active_groups):
33     desc = pd.DataFrame([get_descriptors(s) for s in df["SMILES"]])
34     groups = df[active_groups].reset_index(drop=True)
35     return pd.concat([groups, desc], axis=1)
36
37
38 def load_data(path):
39     df = pd.read_csv(path)
40     group_cols = [c for c in df.columns if c.startswith("Group")]
41     active_groups = [c for c in group_cols if df[c].sum() > 0]
42     X = make_features(df, active_groups)
43     y = df["Tm"]
44     return X, y, active_groups, list(X.columns)
45
46
47 def build_model():
48     return lgb.LGBMRegressor(
49         n_estimators=500,
50         learning_rate=0.05,
51         num_leaves=31,
52         random_state=42,
53         verbose=-1,
54     )
55
56
57 def main():
58     X, y, active_groups, feature_names = load_data(TRAIN_PATH)
59     print(f"분자 {len(X)}개")
60     print(f"결합 피처 {len(feature_names)}개 = Group {len(active_groups)}개 + descriptor 10개")
61
62     model = build_model()
63
64     r2 = cross_val_score(model, X, y, cv=5, scoring="r2").mean()
65     mae = -cross_val_score(model, X, y, cv=5, scoring="neg_mean_absolute_error").mean()
66     print(f"\n교차검증 성능 ->  $R^2 = \{r2:.3f\}$ , MAE = {mae:.2f} K")
67
68     model.fit(X, y)
69     print("최종 모델 학습 완료")
```

```
70
71     joblib.dump({"model": model, "features": feature_names,
72                 "active_groups": active_groups}, MODEL_PATH)
73     print(f"모델 저장 -> {MODEL_PATH}")
74
75
76 if __name__ == "__main__":
77     main()
```

결론

	sample1.csv Complete (after deadline) · 34m ago	39.35848	36.24743	☐
	sample2.csv Complete (after deadline) · 12m ago · 13131	38.55810	34.01232	☐
	sample3.csv Complete (after deadline) · 7m ago · 1	32.96323	29.86225	☐

세 차례의 제출 결과는 예측 성능이 Group 단독(39.36) → descriptor 단독(38.56) → 결합(32.96) 순으로 뚜렷하게 향상되었음을 보여준다. 이는 이번 탐구의 핵심 가설 국소적 작용기 정보(Group)와 분자 전체를 요약한 화학 지표(descriptor)가 서로 다른 정보를 담고 있어 을 결합할 때 상호보완적으로 작용한다는 점을 실제 대회 점수로 확인 하였다.

Group 분석에서는 337개의 작용기 피처를 사용했고 MAE 39.36이 나왔다. descriptor 단독으로는 10개의 피처만 썼는데 오히려 38.56으로 더 나은 결과가 나왔다. 피처 개수는 훨씬 적는데 성능은 더 좋았다.

이 차이가 발생한 원인은 두 피처가 담는 정보의 성격이 다르기 때문으로 추정된다. Group은 분자를 잘게 쪼갠 국소적인 작용기 정보이고, descriptor는 분자량이나 극성처럼 분자 전체를 요약한 값이다. 잘게 쪼갠 337개보다 분자 전체를 요약한 10개가 녹는점 예측에는 더 효율적인 정보였던 것으로 보인다.

실제 값과 비교했을 때 두 모델 모두 평균 40K 안팎의 오차가 있었는데, 이는 녹는점이 분자간 힘뿐 아니라 결정 구조 같은 요인에도 영향을 받기 때문에 분자 구조 정보만으로는 완전히 예측하기 어려웠던 것으로 추정된다.

최종적으로 이 두 개를 합쳤더니 MAE가 32.96으로 크게 낮아졌다. Group의 국소 정보와 descriptor의 전체 요약 정보가 서로 다른 부분을 보완해 준 결과로, 둘을 합쳤을 때 각각 단독으로 썼을 때보다 성능이 확실히 높아졌다.